

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Modelos de Computação

Modelos formais de computação são importantes para a compreensão das dificuldades inerentes de se resolver problemas computacionais.

- ▶ Máquina de Turing
- ▶ λ -cálculo
- ▶ RAM
- ▶ Circuito booleano

Modelos de Computação

Modelos formais de computação são importantes para a compreensão das dificuldades inerentes de se resolver problemas computacionais.

- ▶ Máquina de Turing
 - ▶ λ -cálculo
 - ▶ RAM
 - ▶ Circuito booleano
- } modelos uniformes
- } modelo não uniforme

A especificação de um modelo de computação contempla

1. descrição do dispositivo
(máquina/programa) do modelo,

A especificação de um modelo de computação contempla

1. descrição do dispositivo
(máquina/programa) do modelo,
2. como uma *entrada* é apresentada,

A especificação de um modelo de computação contempla

1. descrição do dispositivo (máquina/programa) do modelo,
2. como uma *entrada* é apresentada,
3. como a máquina computa, e apresenta o resultado,

A especificação de um modelo de computação contempla

1. descrição do dispositivo (máquina/programa) do modelo,
2. como uma *entrada* é apresentada,
3. como a máquina computa, e apresenta o resultado,
4. uma noção de *passo elementar* de computação.

Máquina de Turing

é dada por uma sêxtupla

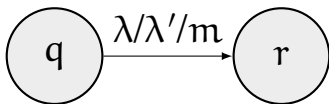
$$(Q, \Gamma, \Sigma, \delta, q_0, q_{\text{para}})$$

- ▶ Q é um conjunto finito de *estados*
- ▶ Γ é o *alfabeto da fita*
- ▶ δ é a *função de transição*
- ▶ $q_0 \in Q$ é o *estado inicial*
- ▶ $q_{\text{para}} \notin Q$ é o *estado final*

A entrada é dada numa fita de leitura/escrita.

A saída é escrita nessa fita.

As transições são os passos elementares.



se a máquina está no estado $q \in Q$ e lê $\lambda \in \Gamma$ na fita, então escreve $\lambda' \in \Gamma$ nessa posição e movimenta-se, de acordo com $m \in \{\leftarrow, \text{—}, \rightarrow\}$ ou uma posição para a esquerda, ou uma para a direita ou permanece na posição que está e, em seguida, entra no estado $r \in Q \cup \{q_{\text{para}}\}$.

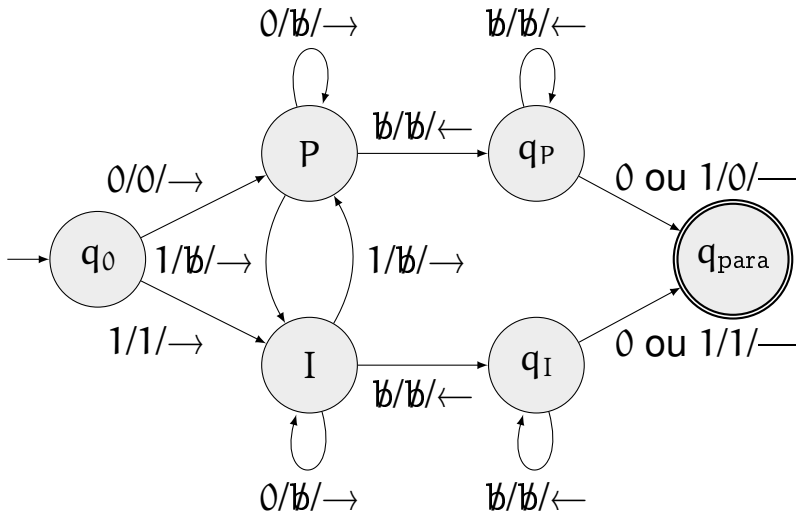
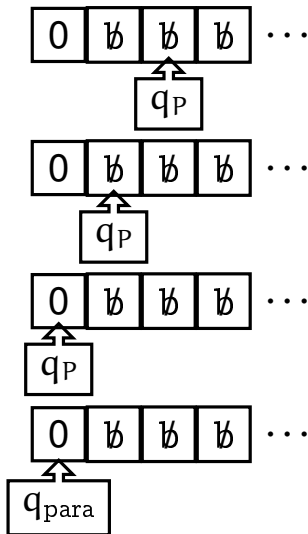
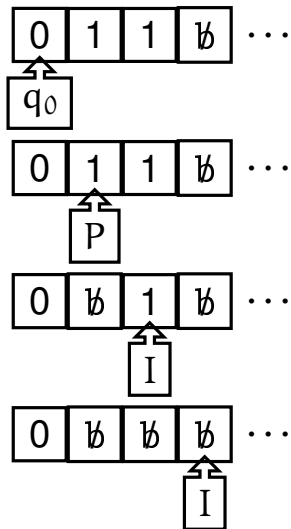


Diagrama de estados da máquina de Turing para o problema da paridade.



fita na execução para 'paridade' com 011.

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Problema computacional

$$(\mathcal{I}, \mathcal{O}, \mathcal{R})$$

- ▶ $\mathcal{I}$ denota o conjunto das instâncias (entradas) do problema,
- ▶ $\mathcal{O}$ o conjunto das respostas (saídas) e
- ▶ $\mathcal{R} \subset \mathcal{I} \times \mathcal{O}$ é uma relação que associa a cada instância uma ou mais respostas.

Uma *solução* de um problema computacional é um dispositivo de computação que computa uma função $A \subseteq \mathcal{R}$.

notação: o dispositivo C com instância $x \in \mathcal{I}$ responde $C(x) \in \mathcal{O}$.

As vezes precisamos distinguir x (entrada/saída) da sua *codificação* $\langle x \rangle$ usando símbolos de algum alfabeto, em geral consideramos bits.

Computabilidade um dado problema tem solução?

Complexidade quanto de um recurso
(tempo/espaco/sorteio) demanda uma
solução?

Complexidade de uma solução

Tempo de execução de pior caso: $T(n)$

Maior número de instruções básicas executadas por um dispositivo dentre todas as instâncias cujas descrição/codificação têm o mesmo comprimento n

Universalidade da MT

O conjunto de todas as máquinas de Turing é enumerável

$$M_1, M_2, \dots$$

É possível estabelecermos uma codificação binária canônica para as máquinas de Turing

Teorema

Existe uma máquina de Turing $\mathcal{U}$ ***universal*** tal que $\mathcal{U}(x, \langle n \rangle) = M_n(x)$. □

O conjunto de todas as máquinas de Turing é enumerável.

O conjunto de todas as funções $\mathbb{N} \rightarrow \{0, 1\}$ não é enumerável.

Logo tem funções que não são computáveis por MT. Dizendo outro modo, existem propriedades de números naturais que não são distinguíveis por programas.

Problema da Parada

decidir se a computação da MT M com uma entrada x termina não é computável.

Supõe existir MT H que recebe (i, x) e decide se a computação $M_i(x)$ termina

$$H(i, x) = \begin{cases} 0 & \text{se } M_i(x) \text{ não termina} \\ 1 & \text{se } M_i(x) \text{ termina} \end{cases}$$

MT $P(k)$:

se $H(k, k) = 0$ **então** **responda** 1;
senão se $\mathcal{U}(k, k) = 1$ **então** **responda** 0;
senão **responda** 1.

MT $P(k)$:

se $H(k, k) = 0$ **então responda** 1;
senão se $\mathcal{U}(k, k) = 1$ **então responda** 0;
senão responda 1.

Se $k := \langle n \rangle$ tal que $M_n = P$, então

MT $P(k)$:

se $H(k, k) = 0$ **então responda** 1;
senão se $\mathcal{U}(k, k) = 1$ **então responda** 0;
senão responda 1.

Se $k := \langle n \rangle$ tal que $M_n = P$, então

$$\mathcal{U}(k, k) = P(k)$$

e

$$P(k) = 0 \text{ se, e só se, } P(k) = 1$$

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Algoritmos eficientes

Modelo RAM tem registradores (memória), $M_0, M_1, \dots$, contador de programa, um conjunto de instruções para transferência de valores entre registradores

INC M_i	Incrementa o conteúdo do registrador i de 1
DEC M_i	Decrementa o conteúdo do registrador i de 1
CLR M_i	Substitui o conteúdo de M_i por 0
$M_i \leftarrow M_j$	Substitui o conteúdo de M_i pelo de M_j
$M_i \leftarrow MM_j$	Substitui o conteúdo de M_i pelo de M_{M_j}
$MM_i \leftarrow M_j$	Substitui o conteúdo de M_{M_i} pelo de M_j
JMP N_i	Atribui N_i ao contador de programa
$M_j \text{ JIFZ } N_i$	Se $M_j = 0$, atribui N_i ao contador de programa
CONTINUE	Continue na próxima instrução, caso exista, ou pare

Um *programa* RAM é uma sequência finita de instruções $N_0, N_1, \dots, N_m$

Equivalência polinomial

Uma MT que executa T transições pode ser simulada por um programa RAM que executa $O(T)$ instruções.

Um programa RAM com palavra de tamanho limitado que executa T instruções pode ser simulado por uma MT que executa $O(T^3)$ transições.

Codificações razoáveis de informação têm tamanho polinomialmente relacionados.

A classe das funções polinomiais é fechada para soma, multiplicação e composição, ou seja, *tempo polinomial* é preservado por práticas comuns de programação.

Computação eficiente é sinônimo de computação feita por um dispositivo com **tempo de execução polinomial** no tamanho da entrada.

O modelo formal é “irrelevante” com respeito a tempo polinomial.

Para simplificar nós dizemos, de modo genérico, **algoritmo de tempo polinomial** e usamos **pseudocódigo**.

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Linguagens

Instâncias e respostas são codificadas usando um alfabeto Σ (podemos fixar $\Sigma = \{0, 1\}$).

Uma *linguagem* L é um conjunto de *strings*/palavras sob Σ .

Uma MT M **decide** L se

$$M(x) = \begin{cases} 0 & \text{se } x \notin L \\ 1 & \text{se } x \in L \end{cases}$$

Exemplo de problema computacional descrito como linguagem:

Uma representação em Σ da fórmula booleana Φ é denotada por $\langle \Phi \rangle$ e a linguagem SAT é o conjunto

$$\text{SAT} = \{ \langle \Phi \rangle : \Phi \text{ é fórmula booleana satisfazível} \}.$$

Decidir se Φ é satisfazível é equivalente a decidir se a palavra $\langle \Phi \rangle$ pertence a linguagem formada pelas representações de fórmulas satisfazíveis.

Se L é linguagem, definindo (abusando de notação) também por L a função

$$L(w) = \begin{cases} 1 & \text{se } w \in L \\ 0 & \text{se } w \notin L \end{cases}$$

a MT M *decide* L se e só se M *computa* L .

Complexidade

Modelos de Computação

Problema computacional

Algoritmos eficientes

Linguagens

Classes de complexidade de tempo polinomial

Modelos probabilísticos

Classes probabilísticas de tempo polinomial

Criptografia

Classes de complexidade de tempo polinomial

A classe $\mathbb{P}$ — *Polynomial time* — é a classe das linguagens decididas por algoritmos de tempo polinomial.

Por exemplo, o problema de decidir se um inteiro positivo é primo está em $\mathbb{P}$.

Não sabemos se SAT pertence a $\mathbb{P}$ pois não conhecemos nenhum algoritmo de tempo polinomial para SAT.

A classe NP — *Nondeterministic Polynomial time* — é a classe das linguagens L para as quais existe um polinômio p e um algoritmo $M(\cdot, \cdot)$ de tempo polinomial tal que toda instância w

1. $w \in L$ então existe uma *entrada auxiliar* $c \in \{0, 1\}^{p(|w|)}$ tal que $M(w, c)$ responde *sim*;
2. se $w \notin L$ então para qualquer c o algoritmo $M(w, c)$ responde *não*.

$\text{SAT} \in \text{NP}$:

M com entrada (Φ, c) verifica em tempo polinomial se c é uma valoração das variáveis da fórmula booleana Φ que torna a fórmula verdadeira ou não.

Um algoritmo do tipo $M(w, c)$ é chamado de **verificador**.

Uma sequência c como acima é chamada de **certificado**.

Se $w \in L$ e $L \in \text{NP}$ então há um certificado curto que certifica tal fato, senão, caso $w \notin L$, não há um certificado curto de que $w \in L$.

$P \subseteq NP$:

se $L \in P$ e D é um algoritmo polinomial para L então
faça M para L que simula D e ignora a entrada
auxiliar. □

Conjectura de Cook–Levin

Principal problema não resolvido da Teoria da Computação e foi formulado independentemente por Stephen Cook e Leonid Levin em 1971

Problema

$$P \neq NP?$$